CEG5101 CA2 · Q&A Bank (Group 5)

How to use. Each question has a short English answer to say out loud (about 20–40 s), a Chinese key point (中文要点) for your own understanding, who answers first (Kaiming = system and detection, Danze = routing and results), and where the evidence is. Answer pattern: claim → one number → where it is (slide, figure, file) → limitation or next step if relevant. If you do not know: *"We didn't measure that directly. What we did measure is …, and it's in …"*. Never invent a number.

Most likely questions (practise first): 1.3, 3.2, 3.4, 4.5, 5.2, 6.1, 6.2, 6.6, 7.4, 8.2, 10.1, 10.3.

Numbers come from `presentation/data/numbers.md` and `report/report.pdf`. R1 = `results/validation-abc`, R2 = `run1-abc`, R3 = `run2-abc`, VM = `vm-delay-abc`.

---

1. SDN and OpenFlow concepts

1.1 What does SDN give you here that a traditional network running OSPF could not? (Danze)

The controller has a global view: per-source rates at the ingress and per-link load, at the same time. So within one 2-second interval it identifies the attacker, drops it on all six switches, recomputes paths and installs them end to end. OSPF only knows link weights and must wait for convergence, and it has no single place to block a source. Our block and reroute happened at the same timestamp, 115.6 s, and a link failure was rerouted in 7 ms. The price is that the controller is a single point of failure and polling costs control traffic.

中文要点：全局视图 + 集中决策 + 一个周期内同时完成检测、封禁、全网改路由；代价是控制器单点故障和轮询开销。

Evidence: slide 3, report Fig. 3, `results/validation-abc/summary.md` events.

1.2 Which OpenFlow version and messages do you use? (Kaiming)

OpenFlow 1.3 with Ryu 4.34. We use FlowMod to install forwarding and drop rules, PacketIn and PacketOut for first packets and ARP, FlowStatsRequest and PortStatsRequest every interval for measurement, and LLDP through Ryu's topology discovery to learn the links.

中文要点：OF1.3；FlowMod / PacketIn / PacketOut / FlowStats / PortStats / LLDP。

Evidence: `controller/integrated_controller.py`.

1.3 Explain your control loop in one sentence. (Kaiming)

Every 2.4 seconds the controller measures (flow and port statistics), detects (per-source EMA threshold), prices (dynamic link cost with an anomaly term) and routes (Dijkstra with hysteresis), so detection and routing share one signal: the links a blocked source used get A = 1.

中文要点：测量 → 检测 → 定价 → 选路，每 2.4 秒一轮，检测结果通过 A=1 进入代价。

Evidence: slide 3.

1.4 Is this reactive or proactive routing? (Danze)

Both. A path is computed reactively when the first packet of a host pair reaches the controller, then the whole path is installed at once. After that the controller re-evaluates every installed flow proactively in every interval and moves it when the cost model says so, without waiting for new packets.

中文要点：首包触发算路（被动），之后每个周期主动重评估、主动改路由。

Evidence: `_handle_ipv4`, `_reroute_all`.

1.5 What is the table-miss entry for? (Kaiming)

Priority 0, match everything, send to the controller. It is installed on every switch when it connects, after the table is cleared. Packets that match no rule reach the controller, which decides what to do.

中文要点：优先级 0 的兜底规则，把没匹配到的包送控制器。

Evidence: `switch_features_handler`.

1.6 What priorities do your rules use, and why? (Kaiming)

Table-miss 0, forwarding 10, drop 5000. The drop rule must win over the forwarding rule for the same packets, so it has a much higher priority; 5000 is the same value the reference blocking code uses.

中文要点：0 / 10 / 5000，封禁规则必须压过转发规则。

Evidence: `PRIO_*` constants.

1.7 Why Ryu and not ONOS or OpenDaylight? (Danze)

The assignment's reference code is Ryu, and Ryu is a small Python framework, so the whole controller is about 800 lines that we can test without a network. The logic itself (cost model, Dijkstra, detector) is in a module with no Ryu dependency, so it would port to another controller.

中文要点：参考代码是 Ryu；逻辑和框架分离，可移植。

Evidence: `controller/sdn_logic.py`.

1.8 How does the controller learn the topology? (Kaiming)

Ryu's LLDP-based discovery (`--observe-links`). The controller waits until all 11 switch-to-switch links are seen in both directions before it forwards anything, because until then it cannot tell host ports from switch ports.

中文要点：LLDP 发现链路，11 条链路都发现后才开始转发。

Evidence: `_refresh_topology`, event `topology_ready`.

2. Topology and MiniEdit (Task 1)

2.1 Describe the topology. (Kaiming)

Six switches, each with one host: hi on si with 10.0.0.i/24, host links 40 Mbps and 15 ms. Eleven switch links of 40 Mbps form a mesh, and each link's delay in milliseconds is its static cost C0, from 5 for s1–s2 to 20 for s5–s6.

中文要点：6 交换机 6 主机，11 条网格链路，时延 = 静态代价。

Evidence: slide 2, report Table 1.

2.2 What is the shortest path from h1 to h6, and the backup? (Kaiming)

s1-s2-s6 with cost 5 + 14 = 19. The next best is s1-s2-s3-s6 with 5 + 10 + 7 = 22, only 3 more, which is why it becomes the detour.

中文要点：19 对 22，只差 3。

Evidence: slide 2, `analysis/expected_paths.py`.

2.3 Which links do the three attacks use? (Kaiming)

Case a, h2 to h3, uses s2–s3. Case b, h2 to h6, uses s2–s6, the link our flow shares. Case c, h5 to h6, takes s5-s3-s6 because 9 + 7 = 16 is cheaper than the direct s5–s6 at 20.

中文要点：a: s2-s3；b: s2-s6（共用）；c: s5-s3-s6。

Evidence: slide 2.

2.4 How did you build it in MiniEdit? (Kaiming)

In MiniEdit on the Mininet 2.3.0 VM: placed six switches and six hosts, drew the eleven switch links and six host links, set every link's bandwidth and delay in the link properties dialog, set each host's IP, added controller c0 as a Remote Controller at 127.0.0.1:6633 and linked it to every switch, enabled only OpenFlow 1.3 in Preferences, saved it as `my_topology.mn`, then pressed Run and ran `pingall`.

中文要点：放节点 → 连线 → 填 40 Mbps 和时延 → 主机 IP → 远程控制器 → 只开 OF1.3 → 保存 → Run → pingall。

Evidence: report Section 2, Figs. 1–2, `miniedit/my_topology.mn`, the MiniEdit recording.

2.5 How do you know the MiniEdit topology matches the given one? (Kaiming)

We compared the saved `.mn` with `ceg_mesh_topo.py`: all 17 links have the same bandwidth and delay and the hosts have the same addresses. In the running network `pingall` reached 30 of 30 pairs and h1 to h6 pinged in about 101 ms, which matches the 98 ms the delays add up to.

中文要点：逐条比对 17 条链路；pingall 30/30；RTT 101 ms ≈ 理论 98 ms。

Evidence: Fig. 2.

2.6 Why do you need a link from c0 to each switch in MiniEdit? (Kaiming)

In MiniEdit a switch only connects to the controllers it is linked to. Without those control links the switches would have no controller and would forward nothing.

中文要点：MiniEdit 里不连控制线，交换机就没有控制器。

2.7 Any problems with MiniEdit? (Kaiming)

Two. In Mininet 2.3.0 under Python 3, File > Save and Export write an empty file because the file is opened in binary mode; changing ‘wb’ to ‘w’ in two places in miniedit.py fixes it. And the exported Level-2 script drops the OpenFlow 1.3-only setting, which still works with Ryu because OVS enables 1.0 to 1.5 by default.

中文要点：Save 会存空文件（wb→w 修复）；导出脚本丢掉 OF1.3 设置。

Evidence: report Section 2.

2.8 Why is the link delay used as the cost? (Danze)

The reference code and the topology file both define the base cost as the TCLink delay, so C0 = delay in milliseconds. That makes the static route the lowest-latency route.

中文要点：题目约定 C0 = 时延。

3. Monitoring and detection (Task 2)

3.1 What exactly do you measure? (Kaiming)

Two things per interval. Flow statistics give each source’s packet rate, counted only at the switch the source is attached to, so a packet is not counted once per hop. Port statistics give each link’s rate from transmitted bytes, taking the larger of the two directions, smoothed with an EMA weight of 0.6.

中文要点：FlowStats 只在入口交换机计数；PortStats 取两个方向较大值，EMA 0.6 平滑。

Evidence: `flow_stats_reply_handler`, `_evaluate`.

3.2 Why an adaptive threshold, max(3b, b + 4σ, 500 pkt/s)? (Kaiming)

A fixed multiple ignores how bursty a source normally is, and a nearly idle source would trip at 3 pkt/s. We keep 3b as the assignment’s rule, add b + 4σ so the threshold follows each source’s own variability, and a 500 pkt/s floor for small bursts. The flood, about 4,700 to 4,800 pkt/s, is far above all three; h1 at about 890 pkt/s has a threshold of about 2,600 and was never flagged.

中文要点：3b 是题目要求；b+4σ 跟随波动；500 是下限。

Evidence: slide 4, report Section 3.2.

3.3 Why compare before updating the baseline? (Kaiming)

The reference updates first. With EMA weight 0.3, the new baseline is 0.7b + 0.3r, and r > 3 times that works out to r > 21b, so the real threshold is 21 times the baseline, and attack samples get learned. We compare first and never learn anomalous samples.

中文要点：推导 r > 3(0.7b+0.3r) ⇒ r > 21b。

Evidence: slide 4, D5.

3.4 Why exclude ICMP echo replies? (Kaiming)

During a ping flood the victim answers at the same rate. If replies were counted, h6 would look like an attacker in case b and be blocked, which would cut h1 to h6 and finish the attack for the

attacker. We match on the ICMP type so replies can be skipped in the per-source count; their bytes still count as link load.

中文要点：否则受害者 h6 会被当成攻击者封掉。

Evidence: D2/D4, `flow_stats_reply_handler`.

3.5 What is the warm-up and why? (Kaiming)

A source is never judged on its first samples: the first 5 samples (about 12 s) only build its baseline. With real link delays the start-up pings take longer, so our experiment runner raises this to 12 samples automatically.

中文要点：前 5 个样本只学不报；有时延时 12 个。

Evidence: report Section 7.4.

3.6 Where does the 1.5–3.8 s detection delay come from? (Kaiming)

The controller samples every 2.4 s: a 2 s poll plus a fixed 0.4 s wait for the replies. Detection needs one or two samples. If the flood starts late in an interval, the first sample only covers part of it and may stay under the threshold, so we wait one more interval. In R1 case b that first partial sample was 491 pkt/s, just under the 500 floor.

中文要点：2.4 秒一次采样，1–2 个样本；第一个样本可能只覆盖部分洪泛。

Evidence: slide 9, backup slide 19.

3.7 How would you detect faster? (Kaiming)

A shorter poll interval, or packet sampling such as sFlow, would cut the delay, at the cost of more control traffic and noisier rate estimates. Bounding how far one sample can move the baseline would also stop a partial sample from raising the threshold.

中文要点：更短轮询或 sFlow；限制单个样本对基线的影响。

3.8 Can a legitimate high-rate flow be blocked by mistake? (Kaiming)

Yes, that is the price of a baseline-relative detector. h1 was safe because its iperf started during its warm-up, so its baseline was learned from it. A flow that starts later at that rate would be flagged. We actually saw this on the delayed VM with the short warm-up: h1 was blocked, which is why the runner now uses 12 samples there. Mitigations: allow-lists, rate-limiting new heavy flows with meters instead of dropping, or requiring two consecutive alarms.

中文要点：会；VM 上真的发生过；缓解办法：白名单、meter 限速、连续两次超阈值。

Evidence: report Section 7.4.

3.9 Did you have any false positives or false negatives? (Kaiming)

In the nine container floods: every attacker was detected and only attackers were blocked, h2 and h5. No legitimate source was blocked. The one false positive we know of is the VM run with the 5-sample warm-up, which we kept as evidence and fixed.

中文要点：9 次洪泛全检出、无误封；唯一误封是 VM warm-up 5 那次。

3.10 Why EMA rather than a sliding window? (Kaiming)

It needs constant memory per source, reacts smoothly, and the reference code uses it. We added an exponentially weighted variance so the threshold can follow each source's burstiness.
中文要点：内存 O(1)，参考代码也用 EMA；加了方差。

3.11 Why 500 pkt/s as the floor? (Kaiming)
Normal sources here send about 1 pkt/s of ping, so 3b alone would be a few packets per second. 500 is far above background traffic and far below the flood. It is tuned for this setup, not derived.
中文要点：远高于背景流量、远低于洪泛；是调出来的。

3.12 Why detect per source IP and not per flow or per destination? (Kaiming)
The mitigation is per source, so detection per source matches it, and the assignment asks to identify the anomalous source. Per-destination aggregation would be needed against distributed attacks, which we list as a limitation.
中文要点：封禁按源，检测也按源；分布式攻击需要按目的地聚合。

4. Blocking and mitigation (Task 2)

4.1 How do you block an attacker? (Kaiming)
A drop rule on all six switches: priority 5000, match IPv4 source address, no actions, which means drop. It is installed in the same interval as the detection.
中文要点：6 台交换机都装 priority 5000、按源 IP 匹配、无动作（丢弃）。

4.2 How did you verify the block worked? (Kaiming)
Three independent sources. tcpdump on the victim: after each block only packets already in flight arrived, within 28 ms, then nothing, so zero attack packets later than 1 s after the block. The attacker's own counter shows it kept sending for the whole 25 s. And the drop rule's packet counter kept rising, 31,253 packets on s2 about 7 s after the block in R1.
中文要点：受害者 tcpdump、攻击者发送计数、drop 规则计数三方互证。
Evidence: slide 9, Fig. 5, `flows/02_b_during.txt`.

4.3 Why drop on all six switches and not only at the ingress? (Kaiming)
For this attacker the ingress alone would do and costs fewer entries. Installing on all switches costs five extra rules per blocked source and covers the case where the attacker moves, another host spoofs its address, or packets are already inside the network.
中文要点：多 5 条规则换稳健性；大网络应只装在边缘。

4.4 How many attack packets got through before the block? (Kaiming)
Between 3,608 and 14,251 per flood, roughly the flood rate times the detection delay. That is the cost of the detection delay, and it is where all the loss happened.
中文要点：3,608–14,251 个，约等于速率 × 检测延迟。

4.5 When do you unblock, and why automatically? (Kaiming)

When the drop rule has been quiet for 10 s, meaning the drop rate is below half the threshold. Automatic unblocking limits the damage of a false alarm and lets the same host attack again in the next case, which our timeline needs. The attacker's normal 1 pps ping does not keep it blocked.

中文要点：drop 速率低于 0.5T 持续 10 秒就解封。

4.6 Blocking by source IP also drops the attacker's legitimate traffic. Is that a problem? (Kaiming)

Yes, while blocked all of h2's traffic is dropped, including its 1 pps ping to h4. We limit it in time with automatic unblocking. Finer options would be to match source plus protocol, or to rate-limit the source with an OpenFlow meter instead of dropping.

中文要点：会误伤攻击者的正常流量；改进：按协议匹配或用 meter 限速。

4.7 What if the attacker pulses: floods, waits to be unblocked, floods again? (Kaiming)

Each pulse pays the detection delay again. Our timeline contains one repeat: h2 attacks in case a, is unblocked, attacks again in case b, and is caught again. We did not test short pulses; exponential back-off on the block time for repeat offenders would handle it.

中文要点：每次都要再付一次检测延迟；改进：重复违规者封禁时间指数增长。

4.8 What if the attacker spoofs its source address, or many hosts attack at a low rate? (Kaiming)

A source-IP drop rule fails against spoofed or random sources, and if every source stays under 500 pkt/s nothing is detected. Then the reroute is the remaining protection, because the marked links stay expensive. A real fix needs per-destination detection and rate limiting; we did not implement it.

中文要点：按源封禁无效；改路由是剩下的防线；需要按目的地检测和限速。

5. Dynamic link cost (Task 3)

5.1 Explain the cost formula. (Danze)

$C = C0 + \alpha \cdot R/Rmax + \beta \cdot A$, with $\alpha = 10$, $\beta = 30$ and $Rmax = 40$ Mbps. R is the measured, smoothed link rate, clipped at the capacity, and A is 1 if the link carries traffic of a blocked source. So a full link adds up to 10, and an attacked link adds 30.

中文要点：静态 + 负载项（最多 +10）+ 异常项（+30）。

Evidence: slide 5, Eq. (1).

5.2 Why $\alpha = 10$ and $\beta = 30$? What happens with a smaller β? (Danze)

$\alpha = 10$ lets a full link add up to 10, enough to matter under congestion without swamping the base costs. β is the attack memory. By intuition β must exceed the detour's extra 3 plus the margin 2, so 5; counting the flow's own load that the what-if adds to the detour, about 7.7 in steady state. With $\beta = 0$ or 2 the flow never moved. $\beta = 30$ leaves a wide margin; it is tuned, not derived.

中文要点：β 门槛直觉 5、稳态约 7.7；β=0/2 不动；30 留余量。

Evidence: slide 13, report Section 7.1.

5.3 Why do you need the anomaly term at all, if the load term already sees congestion? (Danze)

Because the block removes the load. Within a few intervals after the block the flood no longer reaches s2–s6, so the load term falls back. Without β·A the cost would drop and the flow would move straight back onto the attacked link. β·A keeps the cost high while the attacker is still hitting its drop rule.

中文要点：封禁后负载消失，只有 β·A 能"记住"攻击。

Evidence: slide 5, `cost_s2s6` figure.

5.4 Why does s2–s6 stay at 3.14 times C0? (Danze)

After the block the load term decays to almost zero, but A stays 1 while h2 keeps hitting its drop rule, plus a 10 s hold. So the cost is 14 + 0 + 30 = 44, and 44 over 14 is 3.14, exactly as predicted. The other attacked links plateau at (C0 + 30)/C0 too: 4.0, 4.33 and 5.29.

中文要点：(14+30)/14 = 3.14；其他链路同理。

Evidence: slide 11, Fig. 1.

5.5 Why is the peak at detection 3.67 and not 3.14? (Danze)

At the moment of the block the flood load is still in the smoothed rate, so the load term adds about 7 on top of C0 + 30. It decays over the next few intervals to the plateau.

中文要点：检测那一刻负载项还在，之后几个周期衰减到平台。

5.6 Why do plateaus differ between links? (Danze)

β is the same 30 on every link, but the normalised value divides by C0, so a short link rises more: s3–s6 with C0 = 7 reaches 5.29, s2–s6 with C0 = 14 reaches 3.14.

中文要点：β 相同，除以不同的 C0。

5.7 Which links get A = 1? (Danze)

The links on the paths of the blocked source's high-rate flows in the interval that triggered detection. In our runs: s2–s3 in case a, s2–s6 in case b, s3–s5 and s3–s6 in case c, the same in every run.

中文要点：攻击源高速率流实际走过的链路。

5.8 Is a marked link unusable? (Danze)

No, it is only more expensive. If every alternative were even worse, Dijkstra would still use it. That keeps the network connected.

中文要点：只是变贵，不是禁用。

5.9 Why is s1–s2 at 1.54 times C0 the whole time? (Danze)

It carries h1's own 10 Mbps flow, which adds 10 × 10.8/40, about 2.7, to a C0 of 5. That is the load term doing its job; it is the same on both of h1's paths, so it does not change the

choice.

中文要点：h1 自己的负载，两条路径都经过 s1-s2，不影响选择。

5.10 Why is the link rate the larger of the two directions? (Danze)

The links are full duplex and a flood loads one direction. Taking the larger direction gives one number per link that reflects the busier side, which is what matters for congestion.

中文要点：取较忙的方向。

6. Rerouting (Task 4)

6.1 If blocking alone brings the loss to zero, why reroute at all? (Danze)

In our runs blocking does stop the loss, and the β experiment shows rerouting adds no further reduction. Rerouting covers what a source-IP block misses: spoofed or random sources, or new sources still under the threshold, whose traffic would keep crowding the attacked link. It costs 22 instead of 19, about 3 ms one way, and only while the link is marked. It is also what Task 4 asks for.

中文要点：封禁止损，改路由防封禁漏掉的流量；代价小且是临时的。

Evidence: slide 13.

6.2 Case c: the assignment says it floods s5–s6, but you marked s3–s5 and s3–s6 and h1 did not move. Is that a failure? (Danze)

No. With the base costs, h5 to h6 takes s5-s3-s6, 16 against 20, so the flood really used s3–s5 and s3–s6, and those are the links we marked. They are not on h1 to h6, so not moving is correct. To test the assignment's intent we set s5–s6 to cost 8: the attack then used s5–s6, it was caught in 1.5 s, and h1 still stayed, because s5–s6 is not on its path either.

中文要点：按代价攻击走 s5-s3-s6；覆盖实验里也正确。

Evidence: slide 12, backup slide 20.

6.3 How do you prevent route flapping? (Danze)

Three mechanisms. The what-if adds the flow's own rate to links it does not use yet; otherwise its own 10 Mbps makes the current path always look worse. A move needs to be cheaper by a margin of 2. And A is held while the attacker hits its drop rule, plus 10 s. Result: exactly two reroutes per case b in every run, away and back.

中文要点：what-if、margin 2、A 保持。

Evidence: slide 6.

6.4 What is the "what-if" exactly? (Danze)

When we evaluate moving a flow, we build the cost graph with that flow's own rate added to every link it is not already using, and compare the cost if it moved against the cost where it is. Without it, a flow would compare its loaded path against empty alternatives and keep jumping.

中文要点：评估换路时，把本流负载加到新路径上它还没走的链路。

6.5 How do you switch paths without losing packets? (Danze)

Egress first. We install the new path from the last switch back to the first: s6, s3, then s2, the branch point where traffic actually switches, then s1. When s2 switches, s3 and s6 already have rules. Stale rules are deleted afterwards. From 116 s every second showed zero loss, and the return caused no dip.

中文要点：先装出口再装入口，分叉点最后改。

Evidence: slide 6, D10.

6.6 Why does the flow return only about 14 s after the flood? (Danze)

After the flood stops, the drop rate falls within one or two intervals, A is held for 10 s more, and in the next interval the original path is cheaper again: 24.5 against 30.1, so it moves back. The delay is deliberate, so a short pause in the attack does not make traffic bounce.

中文要点：10 秒 hold + 1–2 个采样周期；故意的。

Evidence: slide 10.

6.7 Which flows are rerouted? (Danze)

Every installed host pair is checked every interval, in both directions separately, except flows from a blocked source. In case b both h1 to h6 and the replies h6 to h1 move off s2–s6.

中文要点：所有主机对（双向分别）都检查；被封源的流不处理。

6.8 Why Dijkstra and not multipath or k-shortest paths? (Danze)

The cost model gives one number per link, and the assignment asks for a single best path; Dijkstra with deterministic tie-breaking is simple and predictable. Multipath, for example ECMP or OpenFlow groups, could spread load and react to failure in the data plane, which would be an extension.

中文要点：单一最优路径足够；多路径是扩展方向。

6.9 What if both the main path and the backup were attacked? (Danze)

Both would get +30 and Dijkstra would pick the cheapest remaining path, for example through s4 or s5. If every path were marked, it would use the cheapest marked one rather than disconnect. We did not test simultaneous attacks.

中文要点：选剩下最便宜的；没测过同时攻击。

6.10 How do you handle a link failure? (Danze)

When LLDP reports the link down, the current path’s cost becomes infinite and the same comparison moves the flow. With s2–s6 down for 20 s, h1 to h6 moved within 7 ms; 11 of 71,515 packets were lost, all in that second; it moved back 0.7 s after the link returned. The test also found a bug: with fewer than 11 links the controller fell back to “not ready” and ignored ARP, so we made the ready state stick after the first full discovery.

中文要点：7 ms 改路由、71,515 丢 11；发现并修了 D13 bug。

Evidence: slide 13, report Section 7.3.

7. Experiment methodology

7.1 Describe your experiment. (Kaiming)

One script builds the topology, starts the controller and runs a 255-second timeline: 40 s of normal traffic, then for each case a 25 s flood and 45 s of recovery. Legitimate traffic is 10 Mbps UDP from h1 to h6 plus a ping every 0.5 s; every host also sends a 1 pps background ping so the detector has baselines. The attack is hping3 ICMP flood with 1,000-byte payloads. We repeated the full run three times.

中文要点：40 s 基线 + 3 ×（25 s 洪泛 + 45 s 恢复），重复 3 次。

Evidence: slide 7.

7.2 How do you measure loss and throughput? (Kaiming)

From the iperf server on h6, which reports lost and received datagrams every second, independent of the controller. RTT comes from ping, and attack packets at the victim from tcpdump. The controller's own logs are only used for its decisions and costs.

中文要点：受害端 iperf 每秒报告，独立于控制器。

7.3 Why UDP and not TCP for the legitimate flow? (Kaiming)

UDP sends at a fixed rate, so loss shows up directly as lost datagrams per second. TCP would back off under congestion and hide the loss as lower throughput, which makes the effect of the attack harder to read.

中文要点：UDP 固定速率，丢包直接可见；TCP 会退避掩盖丢包。

7.4 How consistent are the three runs? (Danze)

Paths and event order are identical: only case b reroutes, at the block, exactly twice, and no attack packet arrives later than 1 s after a block. Only the detection delay varies, 1.5 to 3.8 s, depending on where the flood starts within a sampling interval, and case b's mean loss follows it: 3.8 s gives 5.67%, 2.6 s gives 4.20%, 1.5 s gives 1.87%.

中文要点：行为完全一致；只有检测延迟不同，平均丢包随之变化。

Evidence: backup slide 18.

7.5 Why only three runs? Is that statistically enough? (Danze)

Three runs show that the behaviour is the same every time and that the variation is explained by one factor, the detection phase. For tight confidence intervals on loss we would need more runs; the script makes that easy.

中文要点：足以说明行为一致；要精确统计需要更多次。

7.6 Why are floods 25 s and recovery 45 s? (Kaiming)

25 s is long enough to see the plateau and the hold, and 45 s lets the 10 s hold, the unblock and the return finish before the next case, so the cases do not overlap.

中文要点：保证每个 case 完整结束再开始下一个。

7.7 Where did you run the experiments? (Kaiming)

R1 to R3 ran in a container with the OVS userspace datapath and without link delays. We repeated the full run on the official Mininet 2.3.0 VM with the kernel datapath and real link delays: same blocks, same marked links, same reroutes, detection in 1.6 to 3.8 s.

中文要点：容器 R1–R3 + Mininet VM 带时延重跑。

Evidence: report Section 7.4, backup slide 21.

7.8 Are results without link delays valid? (Kaiming)

For routing, yes: decisions use the controller's cost table, port statistics and A, not measured delay, and the 40 Mbps limits were enforced, so congestion and loss are real. Absolute RTT changes: on the VM the baseline RTT was 99.8 ms against the 98 ms the delays predict, and 109 ms on the detour. The VM was software-emulated, so we use it to cross-check behaviour, not for loss figures.

中文要点：路由不依赖实测时延；VM 验证了 RTT。

8. Results and graphs (Task 5)

8.1 Summarise your results in three sentences. (Danze)

Every flood was detected in 1.5 to 3.8 s and blocked on all switches, and no attack packet reached the victim later than 1 s after the block. Only case b, which shares h1's link, rerouted h1 to h6, at the block itself, and it came back about 14 s after the flood. All observed loss fell inside the detection delay; outside it h1 to h6 kept 10.5 Mbps with 0% loss.

中文要点：检测 1.5–3.8 s；只有 b 改路由；丢包全在检测延迟里。

8.2 How should we read a mean loss of about 4% next to a worst second of 57%? (Danze)

The mean is over the whole 25 s flood, 5.67, 4.20 and 1.87%, but the loss is concentrated in the few seconds before detection: 24, 47, 48 and 27% in R1, then zero after the block. So the metrics that matter are the worst second before detection and the detection delay; the mean hides them.

中文要点：平均值稀释了检测前那几秒。

Evidence: slide 10, Fig. 2.

8.3 Why does loss reach 48% before detection? (Danze)

s2–s6 carried about 39 Mbps of flood plus our 10.5 Mbps on a 40 Mbps link, so the queue overflowed and a large share of h1's packets were dropped. How the loss splits between the two flows depends on the queue; we measured it rather than modelled it.

中文要点：链路超载约 50/40 Mbps，队列溢出。

8.4 Why is there no loss in case a? (Danze)

The flood h2 to h3 uses s2–s3, which h1 to h6 does not touch, so h1's path never congests. The controller correctly does not reroute.

中文要点：不共用链路。

8.5 Why is there a small loss in case c even though h1 did not move? (Danze)

Both the flood and h1's flow end at h6, so they share the access link s6–h6. No routing change can avoid the last hop; only blocking helps, and the loss stopped at the block. Worst second 2.5 to 6.1%.

中文要点：共用 h6 的接入链路，最后一跳绕不开。

8.6 Explain the normalised cost graph. (Danze)

Each panel is one link's C/C0 over time. A flat line at 1 means unloaded and unattacked. At each detection the attacked links spike, then settle on the (C0 + 30)/C0 plateau, then fall back about 13 to 15 s after the flood. Links carrying h1's flow sit a bit above 1 because of its own load, and s5–s6 stays at 1 all run, which tells you case c never used it.

中文要点：尖峰 → 平台 → 回落；s5-s6 全程为 1。

Evidence: slide 11, Fig. 1.

8.7 Why is s2–s6 at 1.0 just before the flow returns, and then back to 1.19? (Danze)

When A clears, h1 is still on the detour and the flood is gone, so s2–s6 carries nothing: cost 1.0. Once h1 moves back, its own load returns to s2–s6 and the cost rises to 1.19 again.

中文要点：A 清零时 h1 还在绕行，链路空载。

8.8 Why does throughput show 10.5 Mbps and not 10? (Danze)

iperf sends 10 Mbps of payload; the reported rate includes UDP and IP headers. It is about 890 datagrams per second.

中文要点：包含头部开销。

8.9 Why did RTT rise to about 20 ms in case b? (Danze)

Queueing on the overloaded s2–s6 before detection; the container had no propagation delay, so the baseline was about 0.5 ms. After the block and the reroute RTT returned to normal.

中文要点：检测前排队。

8.10 What does the blocking figure show? (Kaiming)

For each case, packets the attacker sent versus packets that reached the victim. The victim only received packets before the block, proportional to the detection delay; after it the curve is flat.

中文要点：受害者只在封禁前收到攻击包。

Evidence: slide 9, Fig. 5.

8.11 Why do the path costs at the block show 58.8 versus 30.0? (Danze)

Staying on s1-s2-s6 cost 5 + 2.7 (h1's load on s1–s2) + 14 + load + 30 on s2–s6, about 58.8. The detour with the what-if load was about 30.0. 58.8 is far more than 30.0 plus the margin, so the flow moved.

中文要点：留下约 58.8，换走约 30.0，差远超 margin。

8.12 Is it a problem that the detour adds latency? (Danze)

It adds 3 to the path cost, about 3 ms one way with real delays; on the VM the median RTT went from 99.8 to 109 ms on the detour. The controller only accepts it while the link is marked and returns afterwards.

中文要点：多 3 ms 单程，只在攻击期间。

9. Extended experiments

9.1 What did the β experiment show? (Danze)

With β = 0 or 2 the flow never moved, but loss after the block was still zero. So blocking is what stops the loss, and β is what keeps traffic off an attacked link. That clarifies what each part buys.

中文要点：止损靠封禁，β 让流量离开。

Evidence: slide 13, report Table.

9.2 Why did β = 2 not move the flow? (Danze)

Moving needs the detour to be cheaper by more than the margin. The detour is 3 more by base cost, plus the flow's own load in the what-if; β = 2 adds too little to s2–s6 to overcome that, so staying stays cheaper.

中文要点：β 太小，抵不过绕行多出的代价 + margin。

9.3 What did the case c override show? (Danze)

With s5–s6 set to cost 8, the flood used s5–s6, it was detected in 1.5 s, s5–s6 was marked at about 4.75 times C0, and h1 kept its path because it does not use s5–s6. The worst second lost 35%, all on the shared access link to h6, which only blocking can fix.

中文要点：攻击走 s5-s6，h1 仍不动；丢包来自 h6 接入链路。

9.4 What did the link-failure test show and what bug did it find? (Danze)

Same as 6.10: reroute in 7 ms, 11 of 71,515 packets lost, back 0.7 s after restore. It exposed that the controller dropped its ready state when a link disappeared and then ignored ARP; we latched the ready state.

中文要点：7 ms；D13 bug。

9.5 What did the VM re-run add? (Kaiming)

Real link delays and the kernel datapath, as on a standard Mininet VM. The control behaviour was the same as in the container, RTT matched the delays, and it found one more issue: with delays the start-up pings take longer, so the 5-sample warm-up ended before h1's iperf began and h1 was blocked. The runner now uses 12 samples when delays are on.

中文要点：验证真实时延下行为一致；发现 warm-up 问题。

10. Limitations and improvements

10.1 What is the weakest part of your design? (Kaiming)

The detection delay. Every lost packet in our experiments fell inside those 1.5 to 3.8 s; everything after detection took effect within one interval.

中文要点：检测延迟。

10.2 What would you improve with more time? (Kaiming / Danze)

Faster detection with sFlow or a shorter poll; bounding how far one sample moves the baseline; meters to rate-limit instead of dropping; per-destination detection against distributed or spoofed floods; back-off for repeat offenders; and testing on a hardware-virtualised VM with TCP traffic.

中文要点：更快检测、限速代替丢弃、按目的地检测、退避、更真实的环境。

10.3 Are your parameters derived or tuned? (Danze)

Tuned for this topology and these rates: the 500 pkt/s floor, the 10 s hold and $\beta = 30$. The plateau values are derived and match the data exactly, and the β threshold of about 5 to 7.7 is derived; 30 is chosen with a wide margin above it.

中文要点：下限、hold、β 是调的；平台值和 β 门槛是推导的。

10.4 Does it scale to a large network? (Danze)

Polling every switch every 2 s grows with the number of switches and flows, and per-host-pair rules grow quadratically with hosts. For a large network we would aggregate flows, poll edge switches only, use sampling such as sFlow, and install drop rules only at the edge.

中文要点：轮询和规则数会增长；改进：聚合、只轮询边缘、采样。

10.5 What if the controller fails or is attacked? (Kaiming)

The controller is a single point of failure; installed rules keep forwarding, but no new detection or rerouting happens. Flows from new random sources would also cause many packet-ins, a known SDN weakness. Mitigations: controller replication, rate-limiting packet-ins with meters.

中文要点：单点故障；随机源会产生大量 packet-in。

10.6 Could the attacker use the reroute against you? (Danze)

An attacker that triggers floods could push flows onto longer paths, but only for the flood duration plus the hold, and the block stops the flood itself. Because A only marks links the attacker actually used, it cannot mark arbitrary links.

中文要点：影响有限，A 只标记攻击实际走过的链路。

10.7 What about two attacks at the same time? (Kaiming)

Detection and blocking are per source, so two attackers are blocked independently and both sets of links are marked. We did not test it.

中文要点：按源独立处理；没测过。

10.8 How does this compare with real-world DDoS mitigation? (Danze)

It is similar in spirit to remote-triggered black-holing or BGP FlowSpec: detect, then push a drop rule to the edge. Real systems add traffic scrubbing and rate limiting rather than dropping everything from a source.

中文要点：类似 RTBH / FlowSpec。

11. Implementation details

11.1 What do your flow rules match on? (Kaiming)

Forwarding rules match IPv4 source, destination and protocol, plus the ICMP type for ICMP, so echo requests and replies are separate entries and can be counted separately. Drop rules match only the IPv4 source.

中文要点：转发按源/目的/协议（+ICMP 类型）；封禁只按源。

11.2 Do rules time out? (Kaiming)

No, forwarding rules are permanent; the controller deletes or replaces them when it reroutes, and drop rules are deleted on unblock.

中文要点：没有超时，由控制器显式增删。

11.3 How do you avoid broadcast storms in a mesh without spanning tree? (Kaiming)

The controller answers ARP itself when it knows the target, and otherwise delivers the request only to host-facing ports, so broadcasts never cross switch-to-switch links. Packet-ins are ignored until all 11 links are known.

中文要点：ARP 代答，只发主机端口。

Evidence: D11.

11.4 How is a packet's rate computed from the counters? (Kaiming)

Each interval we take the change in the rule's packet counter at the source's ingress switch and divide by the time since the last evaluation. A counter that goes backwards means the rule was recreated, so we use the new value.

中文要点：计数器差值 / 时间间隔。

11.5 What do the cookies do? (Kaiming)

Forwarding rules and drop rules have different cookies, so when the statistics come back we can tell forwarded traffic from dropped traffic and track the drop rate of a blocked source.

中文要点：区分转发规则和封禁规则的统计。

11.6 Why did you rewrite the reference cost table? (Danze)

The reference table has five links for a four-switch topology and conflicts with our mesh: here s1–s4 is 8 and s2–s4 is 6, there is no s3–s4, and seven mesh links are missing. We kept its principle, cost equals link delay, and rewrote the table for all 11 links.

中文要点：参考表对应 4 交换机拓扑，和我们冲突。

11.7 What do the unit tests cover? (Kaiming)

18 tests of the logic module: the cost formula and clipping, Dijkstra paths on the mesh including the three attack paths, the reroute decision, and the detector (warm-up, threshold, not learning anomalous samples).

中文要点：代价、最短路、检测器。

Evidence: `tests/test_sdn_logic.py`.

11.8 Why are the two EMA weights 0.3 and 0.6? (Danze)

The detector's 0.3 comes from the reference code, so a short burst barely moves the baseline. The link-load weight 0.6 favours the newest sample so the cost reacts quickly. We did not run a sensitivity sweep; both are environment variables.

中文要点：0.3 沿用参考代码；0.6 让代价反应快；没做敏感性扫描。

12. Process and teamwork

12.1 Who did what? (both, one sentence each, Kaiming first)

Kaiming: *"I was responsible for the topology and MiniEdit, the controller loop, and anomaly detection and blocking, and I ran the experiments and the demo."* Danze: *"I was responsible for the dynamic link cost, rerouting and recovery, the Task 5 analysis and figures, and the extended experiments."* (Matches slide 23; adjust to what you actually did.)

中文要点：两人各一句，和第 23 页一致。

12.2 How did you work together? (both)

Describe your real process: how you split the tasks, how you checked each other's work, what you rehearsed.

中文要点：如实描述。

12.3 What was the hardest part? (both)

A good honest answer: making detection and routing agree, for example the echo-reply problem, the compare-before-update fix, and the bugs found by the link-failure and VM tests.

中文要点：检测和路由的配合；几次测试发现的 bug。

12.4 Can you show it again, or only case b? (Kaiming)

Yes. `bash demo/live_demo.sh quick` runs case b alone in about two and a half minutes, with the same four panes; the full run with all three cases takes about five to six minutes. If the block does not appear within 60 seconds, we switch to the recording on slide 8.

中文要点：quick 模式 2.5 分钟只跑 case b；60 秒没看到 block 就切录屏。

Evidence: `docs/Live_Demo_Runbook_zh.pdf`.